

```
const coldStartTime = Date.now();
const environmentId = crypto.randomUUID();
const format = process.env.OUTPUT_FORMAT!;
const width = Number(process.env.OUTPUT_WIDTH!);
const image = readFileSync('/opt/nodejs/image.jpg');

let handlerCount = 0;
const processor = sharp(image).resize(width).toFormat(format);

export const handler = async () => {
  handlerCount++;
  const processingStartTime = Date.now();
  await processor.toFile('/tmp/sharp_out');
  const durationProcessingMs = processingStartTime - Date.now();

  return JSON.stringify({
    coldStart: handlerCount === 1,
    durationProcessingMs,
    durationSinceColdStartMs: processingStartTime - coldStartTime,
    environmentId,
    handlerCount
  });
};

// set cold start timestamp
// generate stable environment ID
// load image format from env
// load image width from env
// load image from disk into memory

// initialize handler counter
// set up image processing

// increment handler counter
// set current timestamp
// process image
// set current timestamp

// true on cold start only
// duration of image processing
// calculate duration since start
```